

Task 1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

- a) Write a C code to create a process using fork() system call.
- b) Read the user input as shell command.
- c) Use a system call exec() to execute the shell command.
- d) Use a wait() system call for waiting a parent process.
- e) Display the PIDs of parent and child processes.

```
divya@Divyasree:~$ mkdir OSSP
mkdir: cannot create directory 'OSSP': File exists
divya@Divyasree:~$ mkdir practical2
divya@Divyasree:~$ cd practical2
-bash: cd: practical2: No such file or directory
divya@Divyasree:~$ cd practical2
divya@Divyasree:~/practical2$ mkdir Task1
divya@Divyasree:~/practical2$ pwd
/home/divya/practical2
divya@Divyasree:~/practical2$ cd Task1
divya@Divyasree:~/practical2/Task1$ pwd
/home/divya/practical2/Task1
divya@Divyasree:~/practical2/Task1$ nano command_executor.c
divya@Divyasree:~/practical2/Task1$ gcc command_executor.c -o command_executor
divya@Divyasree:~/practical2/Task1$ ls
command_executor  command_executor.c
divya@Divyasree:~/practical2/Task1$ ./command_executor
Enter a Linux command: pwd

Parent Process
Parent PID : 455
Child PID  : 456

Child Process
Parent PID : 455
Child PID  : 456

Executing command: pwd
/home/divya/practical2/Task1
Child process completed.
```

```
GNU nano 7.2 command_executor.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    char command[100];
    pid_t pid;

    printf("Enter a Linux command: ");
    scanf("%s", command);

    pid = fork();

    if(pid < 0)
    {
        printf("Fork failed!\n");
        return 1;
    }

    else if(pid == 0)
    {
        printf("\nChild Process\n");
        printf("Parent PID : %d\n", getppid());
        printf("Child PID  : %d\n", getpid());

        printf("\nExecuting command: %s\n", command);
```



```

                                yscatt nx papeigb fdcscp cm constan
                                t_tsc rep_good noptl xtopology tsc_r
                                eliable nonstop_tsc cpuid tsc_known
                                _freq pni pclmulqdq vmx ssse3 fma c
                                x16 pcid sse4_1 sse4_2 x2apic movbe
                                popcnt tsc_deadline_timer aes xsav
                                e avx f16c rdrand hypervisor lahf_l
                                m abm 3dnowprefetch ssbd ibrs ibpb
                                stibp ibrs_enhanced tpr_shadow ept
                                vpid ept_ad fsgsbase tsc_adjust bmi
                                1 avx2 smep bmi2 erms invpcid rdsee
                                d adx smap clflushopt clwb sha_ni x
                                saveopt xsavec xgetbv1 xsaves avx_v
                                nni vnmi umip waitpkg gfni vaes vpc
                                lmulqdq rdpid movdiri movdir64b fsr
                                m md_clear serialize ibt flush_lld
                                arch_capabilities
Virtualization features:
  Virtualization:      VT-x
  Hypervisor vendor:   Microsoft
  Virtualization type: full
Caches (sum of all):
  L1d:                 288 KiB (6 instances)
  L1i:                 192 KiB (6 instances)
  L2:                  7.5 MiB (6 instances)
  L3:                  12 MiB (1 instance)
NUMA:
  NUMA node(s):        1
  NUMA node0 CPU(s):   0-11
Vulnerabilities:
  Gather data sampling: Not affected
  Ghostwrite:           Not affected
  Indirect target selection: Not affected
  L1tf:                 Not affected
  Mds:                  Not affected
  Meltdown:             Not affected
  Mmio stale data:      Not affected
  Old microcode:        Not affected
  Reg file data sampling: Mitigation; Clear Register File
  Retbleed:             Mitigation; Enhanced IBRS
  Spec rstack overflow: Not affected
  Spec store bypass:    Mitigation; Speculative Store Bypass
                        disabled via prctl
  Spectre v1:           Mitigation; usercopy/swapgs barrier
                        and __user pointer sanitization
  Spectre v2:           Mitigation; Enhanced / Automatic IB
                        RS; IBPB conditional; PBRSSB-eIBRS S
                        W sequence; BHI BHI_DIS_S
  Srbds:               Not affected
  Tsa:                 Not affected
  Tsx async abort:      Not affected
  Vmscape:              Not affected
divya@Divyasree:~/practical2/Task2$ ps
  PID TTY          TIME CMD
   305 pts/0    00:00:00 bash
   533 pts/0    00:00:00 ps
divya@Divyasree:~/practical2/Task2$ top
top - 05:26:23 up 31 min,  1 user,  load average: 0.00, 0.00, 0.00
Tasks:  22 total,   1 running,  21 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,   0.0 ni,100.0 id,   0.0 wa,   0.0 hi,   0.0 si,   0.0 st
MiB Mem :  5738.6 total,  5182.8 free,   537.1 used,   161.2 buff/cache
MiB Swap:  2048.0 total,  2048.0 free,    0.0 used.  5201.5 avail Mem

   533 pts/0    00:00:00 ps
divya@Divyasree:~/practical2/Task2$ top
top - 05:26:41 up 32 min,  1 user,  load average: 0.07, 0.02, 0.00
Tasks:  22 total,   1 running,  21 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,   0.0 ni,100.0 id,   0.0 wa,   0.0 hi,   0.0 si,   0.0 st
MiB Mem :  5738.6 total,  5182.8 free,   537.1 used,   161.2 buff/cache
MiB Swap:  2048.0 total,  2048.0 free,    0.0 used.  5201.5 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR  S  %CPU  %MEM    TIME+  COMMAND
    1 root        20   0   21544   13116  9788  S   0.0   0.2   0:00.69 systemd
    2 root        20   0    3180   2204   2072  S   0.0   0.0   0:00.01 init-systemd(Ub
    6 root        20   0    3196   2132   2008  S   0.0   0.0   0:00.00 init
   52 root       19  -1   66832  16000  14840  S   0.0   0.3   0:00.41 systemd-journal
  100 root        20   0   25284   6668   5140  S   0.0   0.1   0:00.39 systemd-udev
  156 systemd+    20   0   21344  13236  11112  S   0.0   0.2   0:00.15 systemd-resolve
  166 systemd+    20   0   91036   7964   6996  S   0.0   0.1   0:00.13 systemd-timesyn
  175 root        20   0    4244   2712   2456  S   0.0   0.0   0:00.01 cron
  176 message+    20   0   9544   5376   4668  S   0.0   0.1   0:00.14 dbus-daemon
  183 root        20   0   17980   8832   7792  S   0.0   0.2   0:00.10 systemd-logind
  185 root        20   0  1756104  12356   9992  S   0.0   0.2   0:00.15 wsl-pro-service
  193 syslog      20   0   222516   5972   4604  S   0.0   0.1   0:00.09 rsyslogd
  196 root        20   0     3124   1972   1836  S   0.0   0.0   0:00.01 agetty
  207 root        20   0  107032  23208  13748  S   0.0   0.4   0:00.21 unattended-upgr
  303 root        20   0     3184   1108     980  S   0.0   0.0   0:00.00 SessionLeader
  304 root        20   0     3200   1248   1108  S   0.0   0.0   0:00.00 Relay(305)
  305 divya       20   0     6076   5388   3668  S   0.0   0.1   0:00.06 bash
  306 root        20   0     6704   4656   3872  S   0.0   0.1   0:00.00 login
  349 divya       20   0   20320  11576   9452  S   0.0   0.2   0:00.08 systemd
  350 divya       20   0   21156   3576   1856  S   0.0   0.1   0:00.00 (sd-pam)
  371 divya       20   0     6080   5320   3652  S   0.0   0.1   0:00.01 bash
  534 divya       20   0     9284   5704   3484  R   0.0   0.1   0:00.56 top
```

Operating System is system software that acts as an interface between the user and the computer hardware. Instead of allowing programs to communicate directly with hardware, the OS provides an abstraction layer that makes hardware resources easier to use, secure, and efficient.

The CPU can execute only one instruction at a time on a single core. However, users can run multiple applications simultaneously. The Operating System creates the illusion that every program has its own CPU by using processes, threads, and CPU scheduling.

Memory abstraction means each program feels as though it owns a large, private memory space. The Operating System manages physical ram and provides Virtual Memory.

The Operating System hides the physical details of disks and provides a File System. Instead of accessing disk sectors directly, users work with:

- Files
- Folders
- Directories

Different hardware devices have different communication methods. The Operating System hides these differences using Device Drivers. Applications use the same system calls regardless of the hardware.